TreeMAPPO: Fine-Grained Credit Assignment for Agentic LLM Training via Token-Level Trajectory Branching

Anonymous submission

Abstract

We present TreeMAPPO, a reference-free framework for fine-grained credit assignment in agentic large language model training. Rather than assigning a single scalar reward to an entire reasoning trajectory, TreeMAPPO constructs a rollout tree, estimates segment-level contribution and uncertainty from mixed-success leaves, and converts these estimates into dense token-level training signals. The method is designed for mathematical reasoning agents whose trajectories may interleave natural-language reasoning, fenced Python tool calls, tool responses, and final boxed answers. Compared with methods such as TEMPO that exploit shared prefixes among sampled trajectories, TreeMAPPO actively creates new continuations through guided token-level branching, concentrating exploration at uncertain and model-plausible decision points. Compared with TreeRPO-style tree credit assignment based on empirical outcome comparisons, TreeMAPPO uses a statistically grounded maximum-a-posteriori posterior model that estimates both signed segment contribution and uncertainty before producing token-level advantages. Experiments use a resource-efficient split pipeline that rolls out from the base policy, generates advantage-weighted training data, fine-tunes LoRA adapters, and validates per-epoch checkpoints. Current results show modest but consistent validation gains for Qwen2.5-7B in both tool and no-tool settings, while also revealing sensitivity to learning rate, adapter rank, and validation protocol.

Introduction

Sparse end-of-trajectory supervision makes credit assignment a central bottleneck in reinforcement learning for reasoning agents. When a long rollout contains many reasoning tokens, optional tool calls, and only a terminal correctness judgment, a single trajectory-level reward provides little guidance about which intermediate decisions should be reinforced or suppressed. The difficulty is especially acute in agentic settings, where early errors may be repaired by later reasoning or external tool feedback.

Recent work has begun to exploit shared-prefix and tree structure to densify supervision, including implicit prefix-tree methods, explicit tree-sampling methods, off-policy prefix-aware methods, and fine-grained agentic branching methods (Tran, Yao, and Yu 2025; Yang et al. 2025; Li et al. 2025; Huang, Nguyen, and Zimmer 2025; Wang et al. 2026). These methods show that branching structure exposes local comparisons that are unavailable to flat rollout groups. However, existing approaches often rely on naturally occurring prefix overlap, fixed segment boundaries, offline teacher-generated trees, or heuristic credit estimators that do not explicitly represent uncertainty.

This paper develops fine-grained credit assignment for mathematical reasoning agents through token-level trajectory branching. The central idea is to turn a small set of full rollouts into a structured tree of alternative continuations, then infer which segments are helpful, harmful, or uncertain from the pattern of correct and incorrect descendants. We call the resulting method TreeMAPPO (Tree-based Maximum-A-Posteriori Policy Optimization).

Our implementation uses a lightweight tool-calling protocol: the model receives a question and an optional Python tool specification, emits reasoning in text, may invoke the tool via fenced Python markdown blocks, observes the tool response in context, and is required to terminate with a final answer wrapped in `\boxed{}`. Rollouts are organized into trees, branch points are chosen at token granularity, and segment-level posterior estimates are transformed into token-level training advantages. The experiments in this submission emphasize a split one-shot pipeline: collect rollouts from the base model, construct the training set once for each condition, train adapters for multiple epochs, and validate the resulting checkpoints. This isolates the proposed credit-assignment mechanism while keeping the compute budget tractable.

Our contributions are:

- We formulate fine-grained credit assignment as posterior inference over signed segment contributions in a trajectory tree.
- We introduce a guided branching policy that prioritizes uncertain segments and branching nodes while discouraging over-fragmentation and over-concentration.
- We describe a practical training pipeline that converts rollout trees into non-redundant training trajectories with token-level advantages.
- We report split-pipeline LoRA experiments showing small positive validation gains in Qwen2.5-7B tool and no-tool settings, together with negative results that identify instability at overly aggressive learning rates.

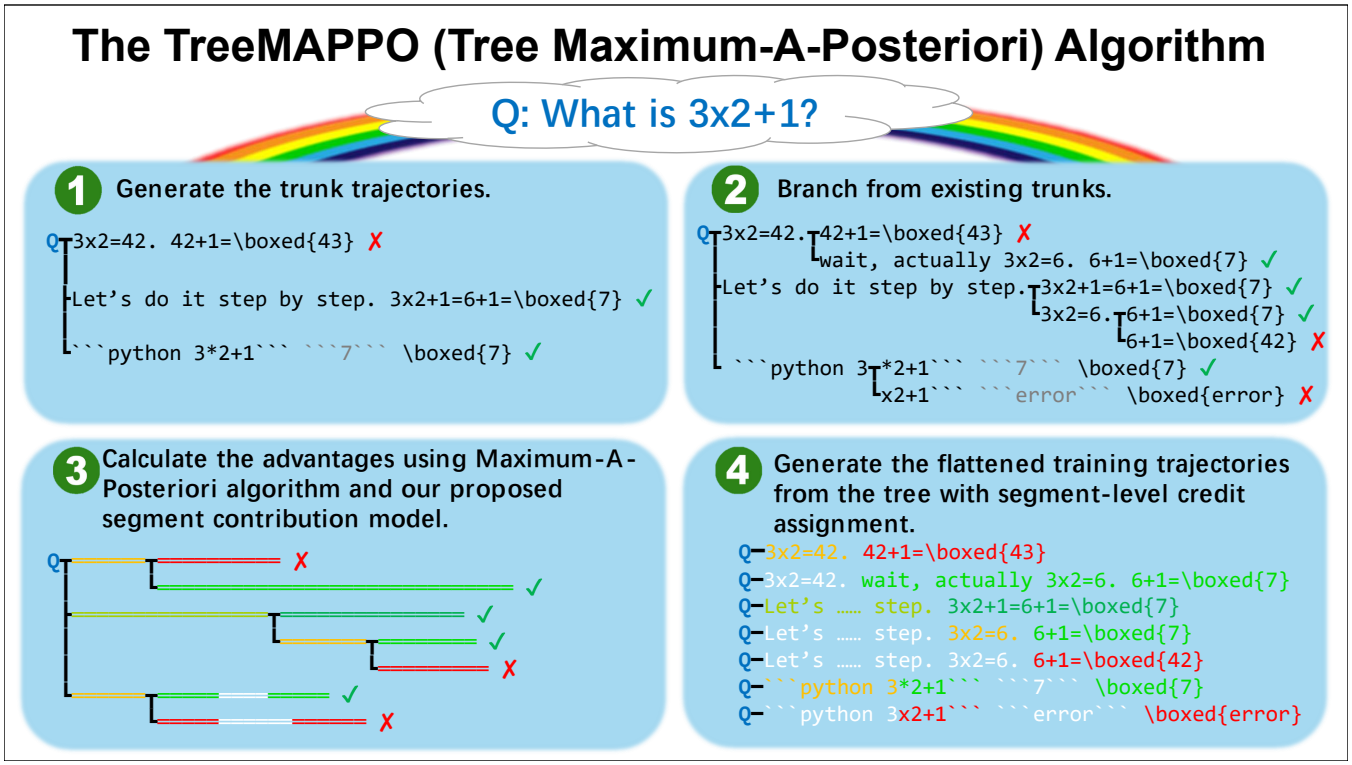

Figure 1: Method overview. TreeMAPPO samples base-policy trajectories, expands selected token-level branch points, infers segment-level contribution and uncertainty from leaf correctness, and converts the resulting posterior estimates into token-level advantages for adapter training.

Related Work

Trajectory-level RL for reasoning. Reinforcement learning with verifiable rewards has become a standard recipe for improving mathematical and agentic reasoning, because final-answer correctness can often be checked automatically. PPO remains a canonical policy-optimization baseline: it stabilizes policy updates with a clipped surrogate objective and obtains token-level advantages through a learned value function (Schulman et al. 2017). GRPO, introduced in DeepSeekMath, removes the critic by normalizing outcome rewards within groups of sampled responses, substantially reducing training complexity while retaining the appeal of verifiable rewards (Shao et al. 2024). The limitation of both trajectory-level and group-relative outcome supervision is the one targeted here: when a long response receives only terminal feedback, many irrelevant tokens are reinforced or suppressed together with the decisive reasoning steps.

Process supervision and verifier-based credit. A complementary line of work makes supervision denser by judging intermediate reasoning steps. Let's Verify Step by Step showed that process supervision can outperform outcome-only supervision on mathematical reasoning by training process reward models from step-level human labels (Lightman et al. 2023). This line establishes the value of fine-grained credit, but it changes the supervision regime: it depends on annotated intermediate labels, learned verifiers, or additional reward-model infrastructure. TreeMAPPO instead keeps the outcome-only setting and infers local credit from the structure of sampled rollouts. The distinction is important methodologically: our goal is not to build a stronger verifier, but to convert sparse terminal correctness into localized training signals without step-level references.

Implicit tree and shared-prefix methods. TEMPO (Tran, Yao, and Yu 2025) converts a group of sampled responses into a prefix tree and adds branch-gated temporal-difference corrections on top of a GRPO-style training loop. Its main appeal is that it preserves a relatively simple rollout interface while extracting token-level signal where natural branching occurs. Our method differs in that we do not rely on incidental prefix overlap alone; instead, we explicitly create new branches at selected token positions, which provides direct control over branching density and lets exploration be driven by posterior uncertainty.

Explicit tree sampling and tree-aware advantages. TreeRPO (Yang et al. 2025) and TreePO (Li et al. 2025) both pursue explicit tree-structured sampling to obtain denser supervision. TreeRPO estimates step-level reward expectations via tree sampling and forms step-level comparison groups without a separate reward model. TreePO emphasizes systems efficiency, shared-prefix reuse, and subgroup-relative advantages over segment-level trees. Our method is closest in spirit to this line, but differs in two impor-

tant respects: first, we branch at arbitrary token positions rather than imposing fixed step or segment partitions; second, we infer segment contribution with a probabilistic MAP model that explicitly represents uncertainty, rather than relying only on win-rate or subgroup heuristics.

Prefix-aware and off-policy tree supervision. Tree-OPO (Huang, Nguyen, and Zimmer 2025) uses teacher-generated MCTS prefixes and staged advantage estimation to create prefix-aware off-policy training groups. By contrast, TreeMAPPO is on-policy and reference-free: the tree is constructed directly from the policy's own rollouts, without offline teacher trees, reward models, or process-level supervision.

Agentic RL and branching granularity. APPO (Wang et al. 2026) studies branching and credit assignment in tool-using agents, arguing that influential decision points are not confined to tool-call boundaries or coarse workflow stages. This observation aligns closely with our setting. The main methodological difference is that APPO uses a branching score and procedure-level advantage scaling, whereas TreeMAPPO combines explicit token-level branching with posterior inference over signed segment contribution and uncertainty.

Tool-augmented mathematical reasoning and benchmarks. Our agent protocol is also related to work on tool-augmented reasoning. PAL and Program-of-Thoughts showed that language models can delegate arithmetic or symbolic computation to Python programs, improving numerical reasoning by separating decomposition from execution (Gao et al. 2022; Chen et al. 2022). ReAct generalized interleaved reasoning and acting, while Toolformer studied how models can learn tool use from limited demonstrations (Yao et al. 2022; Schick et al. 2023). These methods motivate the mixed text–tool trajectories considered here, but they primarily address inference-time tool use rather than training-time credit assignment. Our evaluation setting builds on mathematical reasoning resources such as MATH (Hendrycks et al. 2021) and recent verifiable training corpora such as DeepMath-103K (He et al. 2025); the broader benchmark mix is intended to test whether tree-based credit assignment transfers beyond the distribution used to construct training trajectories.

Problem Setting and Agent Protocol

We study question-answering agents for mathematical reasoning. Each sample provides a problem statement and a ground-truth final answer used only for post hoc correctness judgment. During rollout, the model may produce plain-text reasoning and, when enabled, invoke a Python executor.

Our implementation uses the following protocol.

1. Each prompt contains the problem statement and, when tool use is enabled, a specification that Python can be called by emitting a fenced markdown block.
2. The model may interleave reasoning and tool use.
3. When a fenced Python block closes, generation pauses, the tool executes, and the tool response is appended to the context.
4. The model is instructed to present its final answer in `\boxed{}`.
5. If generation reaches a control handoff without a boxed answer, the framework requests continuation until a boxed answer is produced or rollout termination conditions are hit.

This protocol is consistent across training-data collection and evaluation so that differences across methods are attributable to branching and credit assignment rather than prompt format changes.

Method

Overview

For each question, we first sample a set of complete trunk trajectories. We then expand a rollout tree by branching from selected token positions. Each completed leaf trajectory is judged as correct or incorrect using its extracted final boxed answer. The resulting leaf labels provide supervision for segment-level posterior inference. Posterior estimates are transformed into branching scores for future expansions and into training advantages for model optimization.

Trajectory Tree Construction

In the default configuration for our main tool-using setup, we begin with 4 trunk trajectories and expand to 16 total trajectories. Our experimental plan also includes branch-budget variants with 8 and 32 total trajectories.

Branching is token-level rather than step-level. A new branch can be created in two ways:

- **Segment split:** split an existing reasoning segment at an internal token position and regenerate from that point.
- **Node expansion:** branch again from an existing branching node, increasing its branching factor.

The implementation keeps a fixed decoding temperature within each tree. In the current configs, this value is $T = 0.7$ for trunk generation, continuation, and branch expansion.

MAP Credit Assignment Model

Let ℓ index judged leaves and let i index non-prompt segments. For each leaf ℓ:

- $y_\ell \in \{+1, -1\}$ is the correctness label.
- $x_{\ell,i} \in \{0, 1\}$ indicates whether segment i lies on the path to leaf ℓ.
- Segment i has mean contribution m_i and log standard deviation u_i.

Leaf-level moments are

$$\mu_\ell = \sum_i x_{\ell,i} m_i, \tag{1}$$

$$\tau_\ell = \sqrt{\sum_i x_{\ell,i} \exp(2u_i) + \varepsilon}, \tag{2}$$

$$z_\ell = y_\ell \mu_\ell / \tau_\ell. \tag{3}$$

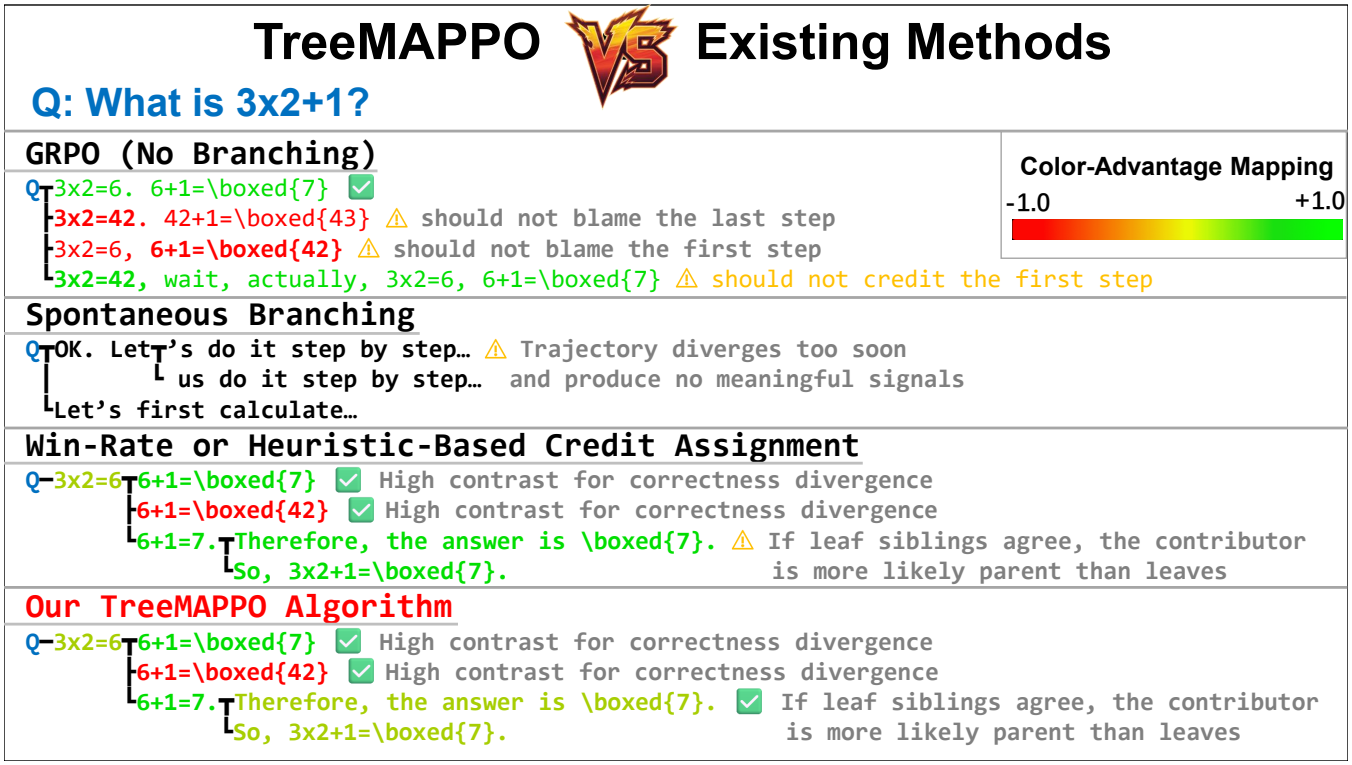

Figure 2: Comparison of credit-assignment structures. Trajectory-level GRPO assigns one normalized outcome signal to every token; TEMPO extracts branch-gated corrections from natural shared prefixes; TreeRPO uses fixed-step tree sampling and empirical child-group rewards; TreeMAPPO combines guided token-level branching with posterior segment-credit inference.

We use a probit likelihood,

$$p(y_\ell \mid \{m_i, u_i\}) = \Phi(z_\ell), \tag{4}$$

where Φ is the standard normal CDF.

Our implementation initializes prior means at zero and optimizes the negative log posterior

$$\mathcal{J} = -\sum_\ell \log \Phi(z_\ell) + \frac{1}{2\sigma_m^2}\sum_i m_i^2 + \frac{1}{2\sigma_u^2}\sum_i u_i^2. \tag{5}$$

The current hyperparameter file uses $\sigma_m = 1.0$, $\sigma_u = 1.0$, $\varepsilon = 10^{-6}$, 120 optimization iterations, and log-standard-deviation clamps in $[-4, 2]$. Optimization is performed with gradient steps and backtracking line search, together with a numerically stable implementation of $\log \Phi$ for large negative margins.

Uncertainty-Aware Guided Branching

Posterior statistics are reused to decide where to branch next. The implementation first computes a signal-to-noise ratio for each segment,

$$r_i = \frac{m_i}{\exp(u_i) + \varepsilon}, \tag{6}$$

then standardizes these values across segments in the same tree,

$$\tilde{r}_i = \frac{r_i - \mathrm{mean}(r)}{\mathrm{std}(r) + \varepsilon}, \tag{7}$$

and maps them to a bounded uncertainty score,

$$s_i = \exp(-\alpha \tilde{r}_i^2), \quad \alpha = 1. \tag{8}$$

Segments whose posterior signal is close to zero receive higher branching priority because they are more uncertain.

For a branching node with parent score s_p and child scores $s_{c_1}, \ldots, s_{c_b}$, the node score is

$$s_{\text{node}} = \frac{b s_p + \sum_{j=1}^{b} s_{c_j}}{2b}. \tag{9}$$

The final implementation-level per-token branching score also includes a model-side alternative-token term derived from the stored top-k logprob candidates. Concretely, for each candidate token position we multiply:

1. a segment or node uncertainty score,
2. a structural penalty, and
3. the relative probability of the best alternative token that differs from the already-sampled continuation.

Candidates whose best alternative has relative probability below 0.1 are discarded.

For node expansion, the structural penalty is an exponential branching-factor penalty,

$$\gamma_{\text{node}}(b) = \exp\big(-k_{\text{node}}(b-1)\big), \tag{10}$$

with $k_{\text{node}} = 0.35$ in the current implementation. For segment splitting, the structural penalty increases with the shorter side of the split relative to average trunk length, discouraging very short fragments.

From Posterior Scores to Training Advantages

After tree construction, each segment receives an unnormalized signed score proportional to $m_i/\exp(u_i)$. The methodology notes propose an additional length-aware rescaling to reduce volatility from very short reasoning spans. The generated training samples then attach token-level advantage values aligned with the input sequence.

The downstream training code consumes aligned token sequences, supervised target tokens, attention masks, token-level advantages, and old token log probabilities. The loss is a clipped policy-ratio surrogate over supervised positions,

$$\mathcal{L}_{\text{train}} = -\frac{1}{|\mathcal{S}|}\sum_{t\in\mathcal{S}} \min\left(\rho_t \tilde{A}_t, \text{clip}(\rho_t, 1-\epsilon, 1+\epsilon)\tilde{A}_t\right), \tag{11}$$

where $\mathcal{S}$ denotes supervised token positions, ρ_t is the ratio between the current token probability and the stored old token probability, and $\tilde{A}_t$ is the token advantage clipped to $[-3, 3]$. This is the implemented optimization interface for injecting segment-level credit into model updates while avoiding the unbounded negative-advantage objective that arises from direct advantage-weighted cross entropy.

Training-Set Construction

Because many tree leaves share prefixes, naively flattening all leaves would over-train shared segments. We therefore use a greedy de-duplication policy.

1. Score each candidate leaf trajectory by average absolute token advantage.
2. Select the strongest candidate.
3. Mark its segments as consumed.
4. Repeat among trajectories that still contain unconsumed learnable segments.
5. Aggregate selected trajectories across trees and prune the tail with a cumulative average-absolute-advantage cutoff.

Selected trajectories are finally ordered by a configurable batching policy. In length-based runs, this improves early memory stability; in by-question runs, it groups positive and negative samples from the same problem so the optimizer sees local contrasts consecutively.

Baselines and Ablations

We consider three primary comparison points.

- **Base model:** no fine-tuning.
- **GRPO-style baseline:** use 8 trunks and 8 total trajectories, yielding multiple complete rollouts without token-level branching in the main tool setup.
- **TreeMAPPO:** guided branching with MAP-based posterior credit assignment.

We also separate two ablation axes that are explicitly encoded in configs and scripts.

- **TEMPO-style branching ablation:** switch the branching policy from guided branching to spontaneous divergence-point branching while keeping posterior-based credit assignment.
- **TreeRPO ablation:** keep guided branching but replace posterior-based segment advantages with a win-rate heuristic.

In our implementation, these ablations are not merely renamed conditions: the TEMPO ablation changes the branching rule, while the TreeRPO ablation changes the advantage-estimation rule. This distinction is important because it cleanly separates exploration structure from credit estimation.

Experimental Setup

Models

We evaluate the following model-condition families.

- Qwen2.5-7B-Instruct with tool use.
- Qwen2.5-7B-Instruct without tool use.
- Qwen3-4B with tool use.
- Qwen3-4B without tool use.
- Gemma-3-4B-IT without tool use.
- Llama-3.1 and Mistral-7B without tool use.

For the main tables, tool-enabled and no-tool variants of the same backbone are treated as separate model-condition groups, since the prompting interface and action space differ.

Datasets

Training and validation are built from DeepMath, MATH, and NuminaMath.

- The training JSONL file contains 5,000 samples per dataset, interleaved across the three in-distribution datasets.
- The validation JSONL file contains 1,000 validation samples per dataset drawn from disjoint index ranges after the training slice.

We also construct an aggregated held-out evaluation JSONL file from six datasets in this order: DeepMath, MATH, NuminaMath, AMC2023, GaoKao Math 2024, and CollegeMath. For datasets with more than 1,000 test rows, 1,000 are sampled with seed 42; for DeepMath, which lacks a dedicated test split in the current pipeline, evaluation rows are drawn from the training split after clearing the reserved training and validation examples.

Training and Evaluation Protocol

The primary experiments use a split one-shot protocol rather than a fully interleaved online reinforcement-learning loop. For each model-condition pair, we run four stages: base-policy rollout collection, training-set generation, adapter training, and checkpoint validation. Rollout and generation are performed once per condition using the base model, after which the generated trajectories and token-level advantages are reused across multiple LoRA training epochs. Validation then evaluates the base model and each trained checkpoint under the same inference protocol.

This protocol is intentionally different from a production online RL pipeline in which each epoch rolls out from the latest policy and immediately regenerates training data. The

split protocol is less expensive, makes infrastructure failures easier to diagnose, and allows fair controlled comparisons among GRPO-style, TreeRPO-style, TEMPO-style, and TreeMAPPO variants under the same rollout budget. When compute permits, the same codebase also supports a full orchestrator mode with repeated validation rollout, training rollout collection, training-set generation, and training across policy updates; those runs are treated as confirmatory rather than the main experimental path.

The training protocol uses the following recurring settings:

- 5–10 one-shot training epochs, depending on the run budget.
- cumulative average-absolute-advantage cutoff of 0.5 for training-set retention.
- time-budgeted training within each epoch, with summaries reporting both samples trained and cumulative dataset-pass coverage.
- LoRA-based fine-tuning for the primary experiments.
- fixed rollout temperature 0.7.

The primary evaluation metric is pass@1 accuracy. We report aggregate accuracy together with per-dataset accuracy, and include confidence intervals when multiple independent runs are available.

Results

This section reports completed measurements available at submission time. Because the experiments use a split one-shot protocol and several longer pipelines are still queued, we emphasize conservative conclusions: the current evidence supports modest validation improvements and useful stability diagnostics, but not yet a definitive claim of broad benchmark superiority.

Validation Accuracy Trends

Table 1 summarizes completed validation runs. "Best" is the best validated trained checkpoint, and gains are measured against the same run's base checkpoint. All listed training runs use single-GPU LoRA fine-tuning.

For Qwen2.5-7B, both GRPO and TreeMAPPO improve over their base checkpoints in the no-tool and tool settings. The magnitude is modest—roughly 1.6–2.2 percentage points in the best validated checkpoint—and the trends are not monotonic across epochs. We therefore treat these as evidence that the split pipeline can produce useful training signal, not as evidence that the method has saturated or consistently dominates all baselines.

Held-Out Serious Test

Table 2 reports the completed held-out test currently available for the Qwen2.5-7B no-tool condition. This test uses a broader six-dataset mixture than the validation curves. At the time of writing, the corresponding GRPO held-out test is not yet complete, so this table is included as a diagnostic rather than as the primary comparison.

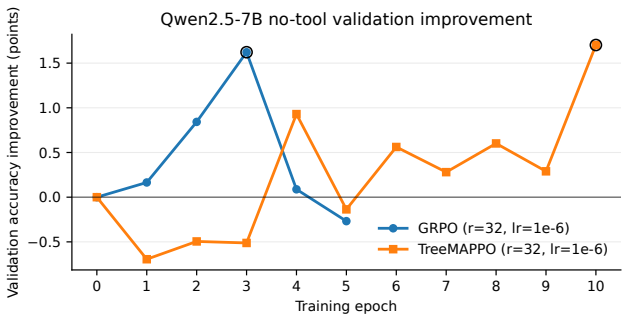

Figure 3: Validation accuracy improvement over the base checkpoint for Qwen2.5-7B in the no-tool setting. Both lines use LoRA rank 32 and learning rate 10^{-6}; the y-axis reports percentage-point change relative to each method's epoch-0 validation accuracy.

Queued branch-budget study
x-axis: total trajectories / leaves in tree (8, 16, 32)
y-axis: downstream pass@1 on validation or aggregated test
show mean and confidence interval bands when repeated runs are available

Figure 4: Branching-budget sensitivity study planned for queued Qwen2.5-7B no-tool jobs.

Ablation Study

The main ablation plan separates branching policy from credit estimation. The TEMPO-style condition changes the branching policy, while the TreeRPO-style condition changes the segment-credit estimator. Only the TreeRPO-style no-tool run is complete so far; TEMPO and branch-budget runs remain queued.

Stability Diagnostics

Several completed runs are negative results that shape the current claims. Qwen2.5-7B TreeMAPPO at LoRA rank 32 with learning rate 5×10^{-5} collapses from a base validation average of 0.6580 to a best trained checkpoint of only 0.2823. Qwen3-4B GRPO at the same high learning rate also degrades sharply, with trained checkpoints below the 0.7055 base. Mistral-7B at rank 16 and learning rate 5×10^{-5} shows an initial epoch-1 gain but then collapses to near-zero validation accuracy. These outcomes motivate the low learning rate used in Table 1 and support presenting adapter-rank and learning-rate sensitivity as central empirical findings rather than implementation details.

Branching-Budget Sensitivity

Our experimental plan includes 8-, 16-, and 32-trajectory variants for the Qwen2.5-7B no-tool setting. These runs are queued and will be used to test whether downstream performance improves with larger rollout trees.

Model and condition	Method	Rank	LR	Epochs validated	Base	Best	Gain
Qwen2.5-7B no-tool	GRPO	32	10^{-6}	5	0.6691	0.6853	+0.0162
Qwen2.5-7B no-tool	TreeMAPPO	32	10^{-6}	10	0.6761	0.6931	+0.0170
Qwen2.5-7B + tool	GRPO	32	10^{-6}	5	0.6412	0.6580	+0.0167
Qwen2.5-7B + tool	TreeMAPPO	32	10^{-6}	5	0.6349	0.6569	+0.0219
Qwen3-4B no-tool	GRPO	16	10^{-6}	5	0.7061	0.7203	+0.0143
Qwen3-4B no-tool	GRPO	32	10^{-6}	5	0.7141	0.7251	+0.0110
Mistral-7B no-tool	GRPO	32	10^{-6}	5	0.2056	0.2345	+0.0289

Table 1: Completed validation results. Values are pass@1 averages on the validation mixture. The Qwen3-4B and Mistral rows are preliminary model-family probes; the main controlled comparison is Qwen2.5-7B at LoRA rank 32 and learning rate 10^{-6}.

Setup	DeepMath	MATH	NuminaMath	AMC2023	GaoKao Math 2024	CollegeMath	Avg.
Base	0.614	0.737	0.640	0.500	0.654	0.753	0.650
TreeMAPPO, epoch 10	0.623	0.737	0.630	0.500	0.577	0.759	0.638
Change	+0.009	0.000	-0.010	0.000	-0.077	+0.006	-0.012

Table 2: Completed held-out serious test for Qwen2.5-7B no-tool TreeMAPPO at LoRA rank 32 and learning rate 10^{-6}. The result improves DeepMath and CollegeMath slightly but lowers the six-dataset average, mainly due to the GaoKao Math 2024 subset. We use this table to qualify the validation gains and motivate stronger final evaluation with multiple rollouts per question.

Qwen2.5-7B no-tool	Base	Best	Gain
GRPO	0.6691	0.6853	+0.0162
TreeRPO-style	0.6737	0.6814	+0.0077
TreeMAPPO	0.6761	0.6931	+0.0170

Table 3: Partial no-tool ablation at LoRA rank 32 and learning rate 10^{-6}. The TreeRPO-style run uses heuristic win-rate credit in place of posterior segment credit.

Discussion

The main conceptual benefit of TreeMAPPO is that it turns sparse trajectory-level outcomes into dense supervision without requiring reference solutions or manually annotated intermediate steps. The completed validation results show that this signal can improve Qwen2.5-7B adapters in both tool and no-tool settings, with gains comparable to or slightly larger than the GRPO-style baseline under the low-learning-rate rank-32 configuration. The held-out serious test is more mixed, however: the best no-tool TreeMAPPO checkpoint improves DeepMath and CollegeMath but lowers the aggregate across six datasets. This gap between validation and held-out test performance is important evidence that final claims should rely on broader evaluation, not only per-epoch validation curves.

Several practical design choices are worth emphasizing.

- Branch selection is not based on posterior uncertainty alone; it also depends on the model's alternative-token probability mass, which makes branching more realistic and less likely to chase implausible continuations.
- The training loss uses a clipped policy-ratio surrogate with token-level advantages produced by the training-set generator. This keeps the optimization interface close to standard policy-gradient practice while preserving the central contribution: tree-based posterior credit assignment.
- Tool-enabled and no-tool settings are analyzed separately because the credit-assignment problem differs materially once trajectories can call Python and recover from arithmetic mistakes.

Limitations and Future Work

The present system has several important limitations.

- Tree construction requires more inference than flat rollout collection, so the method trades additional sampling cost for denser supervision.
- The main experiments use a split one-shot pipeline rather than fully online interleaved rollout and training. This is a practical choice driven by compute limits, queue latency, and debugging efficiency; it means the reported experiments primarily test the quality of the generated credit-assignment data, not the full closed-loop dynamics of repeated policy improvement.
- Current gains are modest and validation trends are noisy. The strongest completed Qwen2.5-7B validation gains are around two percentage points, and the held-out serious test for the no-tool TreeMAPPO checkpoint is mixed.
- Several longer and more controlled experiments are still queued, including Qwen3-4B tool training, Qwen2.5-7B branch-budget sweeps, TEMPO-style ablations, and extended Qwen3-4B no-tool validation. These results are needed before making broad scaling claims.
- The MAP credit model intentionally uses a simple additive segment-contribution assumption. This makes inference tractable and interpretable, but it cannot represent all higher-order interactions among distant reasoning segments.

- Correctness is judged from extracted final answers, so the approach inherits the limitations of answer extraction and exact-answer comparison on mathematical benchmarks.
- The tool environment is deliberately lightweight. Richer agent tools may require additional safety checks, state tracking, and branch-selection criteria.

Conclusion

We presented TreeMAPPO, a trajectory-tree method for fine-grained credit assignment in agentic LLM training. The core idea is to branch reasoning trajectories at token granularity, infer segment-level contribution and uncertainty from mixed-success descendants, and convert those estimates into training signals for subsequent optimization. Completed split-pipeline experiments show modest Qwen2.5-7B validation gains in tool and no-tool settings and identify important stability constraints for LoRA rank and learning rate. The current evidence supports TreeMAPPO as a practical mechanism for producing localized training signal from sparse terminal rewards, while leaving full online training and broader held-out confirmation as necessary next steps.

References

Chen, W.; Ma, X.; Wang, X.; and Cohen, W. W. 2022. Program of Thoughts Prompting: Disentangling Computation from Reasoning for Numerical Reasoning Tasks. *arXiv preprint arXiv:2211.12588*.

Gao, L.; Madaan, A.; Zhou, S.; Alon, U.; Liu, P.; Yang, Y.; Callan, J.; and Neubig, G. 2022. PAL: Program-aided Language Models. *arXiv preprint arXiv:2211.10435*.

He, Z.; Liang, T.; Xu, J.; Liu, Q.; Chen, X.; Wang, Y.; Song, L.; Yu, D.; Liang, Z.; Wang, W.; Zhang, Z.; Wang, R.; Tu, Z.; Mi, H.; and Yu, D. 2025. DeepMath-103K: A Large-Scale, Challenging, Decontaminated, and Verifiable Mathematical Dataset for Advancing Reasoning. *arXiv preprint arXiv:2504.11456*.

Hendrycks, D.; Burns, C.; Kadavath, S.; Arora, A.; Basart, S.; Tang, E.; Song, D.; and Steinhardt, J. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *Advances in Neural Information Processing Systems*.

Huang, B.; Nguyen, T.; and Zimmer, M. 2025. Tree-OPO: Off-policy Monte Carlo Tree-Guided Advantage Optimization for Multistep Reasoning. ArXiv preprint.

Li, Y.; Gu, Q.; Wen, Z.; Li, Z.; Xing, T.; Guo, S.; Zheng, T.; Zhou, X.; Qu, X.; Zhou, W.; Zhang, Z.; Shen, W.; Liu, Q.; Lin, C.; Yang, J.; Zhang, G.; and Huang, W. 2025. TreePO: Bridging the Gap of Policy Optimization and Efficacy and Inference Efficiency with Heuristic Tree-based Modeling. arXiv:2508.17445.

Lightman, H.; Kosaraju, V.; Burda, Y.; Edwards, H.; Baker, B.; Lee, T.; Leike, J.; Schulman, J.; Sutskever, I.; and Cobbe, K. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050*.

Schick, T.; Dwivedi-Yu, J.; Dessì, R.; Raileanu, R.; Lomeli, M.; Zettlemoyer, L.; Cancedda, N.; and Scialom, T. 2023. Toolformer: Language Models Can Teach Themselves to Use Tools. *arXiv preprint arXiv:2302.04761*.

Schulman, J.; Wolski, F.; Dhariwal, P.; Radford, A.; and Klimov, O. 2017. Proximal Policy Optimization Algorithms. arXiv:1707.06347.

Shao, Z.; Wang, P.; Zhu, Q.; Xu, R.; Song, J.; Bi, X.; Zhang, H.; Zhang, M.; Li, Y. K.; Wu, Y.; and Guo, D. 2024. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*.

Tran, H.; Yao, Z.; and Yu, H. 2025. Exploiting Tree Structure for Credit Assignment in RL Training of LLMs. TEMPO, arXiv:2509.18314.

Wang, X.; Ma, Z.; Wang, Y.; Ji, Y.; Yang, S.; Chen, G.; Wang, P.; and Chu, X. 2026. APPO: Agentic Procedural Policy Optimization. arXiv:2606.12384.

Yang, Z.; Guo, Z.; Huang, Y.; Liang, X.; Wang, Y.; and Tang, J. 2025. TreeRPO: Tree Relative Policy Optimization. arXiv:2506.05183.

Yao, S.; Zhao, J.; Yu, D.; Du, N.; Shafran, I.; Narasimhan, K.; and Cao, Y. 2022. ReAct: Synergizing Reasoning and Acting in Language Models. *arXiv preprint arXiv:2210.03629*.